

A PROJECT BASED SEMINAR REPORT

on

RAPtwin

A Digital Twin Simulation Platform for Distributed Systems

Submitted towards the
Partial Fulfilment of the Requirements of

**Bachelor of Technology in
Artificial Intelligence and Data Science**

By

Riaan Masoud Attar Seat No: T231311002

Under the guidance of

Prof. K. K. Deshmukh



Department of Artificial Intelligence and Data Science
K. K. WAGH EDUCATION SOCIETY'S
K. K. WAGH INSTITUTE OF ENGINEERING EDUCATION AND RESEARCH
Hirabai Haridas Vidyanagari, Amrutdham, Panchavati, Nashik, Maharashtra 422003
(An Autonomous Institute affiliated to Savitribai Phule Pune University)
A. Y. 2024-2025 Sem II



Department of Artificial Intelligence and Data Science
K. K. WAGH EDUCATION SOCIETY'S
K. K. WAGH INSTITUTE OF ENGINEERING EDUCATION AND RESEARCH
(An Autonomous Institute affiliated to Savitribai Phule Pune University)

CERTIFICATE

This is to certify that the seminar report entitled

RAPtwin

A Digital Twin Simulation Platform for Distributed Systems

being submitted by

Riaan Masoud Attar Seat No: T372311002

is a bonafide work carried out by student under the supervision of Prof. K. K. Deshmukh and it is submitted towards the partial fulfilment of the requirement of Bachelor of Technology (Artificial Intelligence and Data Science) during academic year 2024-2025.

Prof. K. K. Deshmukh
Internal Guide
Department of
Artificial Intelligence and Data Science

Head,
Department of
Artificial Intelligence and Data Science

Abstract

Researchers and engineers managing modern distributed computing fabrics (such as Edge, Multi-Cloud, and Hybrid environments) often struggle with complex job scheduling, siloed telemetry, and unpredictable hardware failures. Evaluating new, intelligent scheduling policies directly on production infrastructure is highly risky, expensive, and prone to catastrophic failures due to dynamic thermal conditions or link outages. This project introduces RAP twin, a high-fidelity digital-twin and planning sandbox that automates and optimizes job placement, scheduling, and performance analytics for distributed systems in order to address these challenges. The tool utilizes a multi-strategy planning engine, predictive analytics, and Chaos Engineering to intelligently orchestrate heterogeneous workloads across distributed nodes. The system guarantees resilient operations and improved visibility by continuously merging static hardware descriptors (YAML) with streaming live telemetry via a bidirectional REST API to maintain a unified state machine. Automated self-healing controllers, deterministic fault-injection, and dry-run evaluations are essential characteristics which reduce the risk of real-world deployment failures and increase scheduling efficiency. The platform achieves robust policy optimization by combining pluggable scheduling algorithms (ranging from greedy and cheapest-energy to federated and RL-Markov models) alongside Monte-Carlo simulation tooling for large-scale benchmarking under uncertainty. Complex distributed states are managed by a Python-based backend (Flask API), and the simulated output is effectively visualized through a real-time web dashboard rendering topology maps, predictive health tracking, and job histories. The tool is perfect for organizations leveraging edge computing, smart manufacturing, and multi-cloud architectures because it allows for safe, closed-loop simulation and bulk policy evaluation on a local workstation. By offering an automated, scalable, and intelligent emulation sandbox, this project seeks to transform how distributed scheduling policies are tested, eventually allowing organizations to statistically guarantee workload resilience before deploying to expensive physical infrastructure.

Keywords: *Distributed Systems, Digital Twin, Task Scheduling, Chaos Engineering, Predictive Analytics, Monte-Carlo Simulation, Edge Computing, Python, Flask API.*

Acknowledgement

I would like to thank my project guide, Prof. K. K. Deshmukh, for her guidance and support. I will forever remain grateful for the constant support and guidance extended by my guide, in making this project phase I report. Through our many discussions, she helped me to form and solidify ideas. The invaluable discussions I had with her and my team colleagues, the penetrating questions he has put to me and the constant motivation, has all led to the development of the ideas presented in this project phase I report.

I would also like to thank my wonderful colleagues, parents and friends for listening my ideas, asking questions and providing feedback and suggestions for improving my ideas.

Riaan Masoud Attar

Contents

1	Introduction	1
1.1	Project Idea	1
1.2	Motivation	2
1.3	Literature Survey	4
2	Problem Definition and scope	7
2.1	Problem Statement	7
2.1.1	Goals and objectives	7
2.1.2	Assumptions and Scope	8
3	Methodology	10
3.1	Methodology	10
4	Outcome	15
4.1	Type of Project	15
5	Project Plan	17
5.1	Project Timeline	17
5.2	Team Organization	17
5.2.1	Team Structure	18
6	Experimental setup	20
6.1	Data Set	20
6.2	Technology Used	20
6.3	Performance Parameters	21
7	PO & PSO Mapping	23
7.1	Program Outcome (PO) Mapping	23
7.2	Program Specific Outcome (PSO) Mapping	25
8	Summary	27

List of Figures

3.1	RAPtwin - Block Diagram	11
3.2	RAP twin - Simulation and Planning Pipeline	14

List of Tables

5.1	Project Timeline	17
7.1	Program Outcomes (PO) Mapping – RAPtwin	25
7.2	Program Specific Outcomes (PSO) Mapping – RAPtwin	26

Chapter 1

Introduction

1.1 Project Idea

The world today is rapidly advancing into the era of 5G/6G and mobile edge computing, where distributed computing fabrics are becoming highly dynamic and complex. Network engineers and researchers face massive challenges dealing with heterogeneous hardware, siloed telemetry, and varying link reliability. Testing new, intelligent job scheduling policies directly on production infrastructure is not only expensive but also highly risky, as it is prone to catastrophic failures due to unpredictable link outages or thermal throttling. That is where RAP twin comes in.

RAP twin is a high-fidelity digital-twin and planning sandbox designed to automate, optimize, and simulate job placement and scheduling across distributed systems. Instead of risking live infrastructure, this tool provides a safe, self-contained emulation ecosystem that perfectly mirrors physical networks. By continuously merging static hardware descriptors with streaming live telemetry via a bidirectional REST API, RAP twin allows researchers to experiment with cutting-edge offloading strategies in a closed-loop environment.

The main technologies and features that power RAP twin include:

- **Multi-Strategy Planning Engine:** For dynamically optimizing task offloading. RAP twin features a pluggable planning engine supporting baseline greedy and cheapest-energy models, as well as advanced AI-driven Reinforcement Learning (RL-Markov) and Federated models to meet strict latency, cost, and energy targets.
- **Chaos Engineering & Monte-Carlo Tooling:** For evaluating system resilience under real-world uncertainty. The platform can inject deterministic network noise, link jitter, and thermal derating, allowing users to orchestrate thousands of Monte-Carlo trials to statistically validate how scheduling policies survive unpredictable edge conditions [7-9].
- **Predictive Analytics & Unified State Management:** To ensure real-time synchronization between the physical and virtual realms. Background controllers continu-

ously ingest live telemetry via an endpoint to forecast node utilization, failure windows, and dynamically manage resources [4, 10, 11].

Ultimately, the tool aims to bridge the gap between complex theoretical machine learning models and practical edge emulation. The optimized placement plans and performance analytics produced by RAP twin can be visualized in real-time through a web dashboard or exported as detailed statistical reports (e.g., CSV, JSON, Azure DTDL formats), offering versatile benchmarking and dry-run evaluations for various business and research needs.

With RAP twin, organizations and researchers can statistically guarantee workload resilience and Service Level Agreements (SLAs) before deploying to expensive physical infrastructure. Whether in the context of Industry 4.0 smart manufacturing, Internet of Vehicles (IoV), or next-generation multi-cloud telecommunications, RAP twin offers organizations the tools they need to safely modernize their distributed scheduling architectures

1.2 Motivation

In the rapidly advancing era of 5G/6G and mobile edge computing, managing distributed computing fabrics has become a highly dynamic and complex challenge. While next-generation networks promise unprecedented connectivity, network engineers and researchers struggle with managing heterogeneous hardware, siloed telemetry, and varying link reliability. For example, traditional, static computation offloading strategies fail to adapt to unpredictable environmental conditions, making it extremely difficult to maintain a unified, predictive view of how a workload will perform on a specific edge node. Furthermore, testing new, intelligent scheduling policies directly on live production infrastructure is expensive, risky, and prone to catastrophic failures due to unpredictable link outages or thermal throttling.

The motivation behind the development of RAP twin is to solve these pervasive problems in distributed edge orchestration. The tool is designed to provide a safe, high-fidelity digital-twin and planning sandbox, allowing organizations to automate and optimize job placement, scheduling, and performance analytics with minimal risk. Here are the key motivating factors that led to the creation of RAP twin:

1. **Bridging the Gap for Intelligent Scheduling:** While academic literature proposes highly valuable, theoretical Multi-Agent Deep Reinforcement Learning (MADRL) and Federated models for task offloading, there is a lack of practical tools to implement them. RAP twin aims to unlock the potential of these advanced algorithms by providing a multi-strategy planning engine that safely tests policies targeting strict latency, cost, and energy goals.
2. **Risk Mitigation and Cost Savings:** Evaluating intelligent computation offloading strategies directly on physical infrastructure is a labor-intensive and risky process that can disrupt actual services. By automating closed-loop simulations locally, RAP twin

significantly reduces the financial and operational risks associated with testing new policies, freeing up engineers to focus on safe, offline policy sweeps.

3. **Resilience and Chaos Engineering:** Edge computing environments are notoriously riddled with unpredictable errors—node failures, link jitter, and dynamic thermal derating are all too common. RAP twin’s Chaos Engine systematically injects deterministic faults into the network, ensuring that the scheduling algorithms can adapt and reroute intelligently to provide robust, reliable outputs.
4. **Unified State and Predictive Analytics:** In modern edge networks, data integrity and synchronized telemetry are critical. RAP twin helps ensure that live streaming telemetry is continuously merged with static hardware descriptors via a bidirectional REST API (`/observe`). This unified state machine automatically computes predictive analytics like utilization forecasts and failure windows, making it easier to meet stringent operational Service Level Agreements (SLAs).
5. **Data-driven Policy Benchmarking:** In the 5G era, statistical data is key to making informed architectural decisions. RAP twin provides Monte-Carlo simulation tooling that allows organizations to orchestrate thousands of trials under uncertainty. This transforms unpredictable edge behavior into usable statistical insights, driving smarter business decisions by proving which policies best reduce latency and risk.
6. **Scalability for Distributed Systems:** Unlike traditional, localized evaluation methods, RAP twin is designed to handle large-scale distributed computing fabrics. Whether it is generating synthetic profiles for CPU/GPU-heavy workloads or evaluating extensive multi-cloud topologies, the tool can process complex node descriptors and seamlessly scale to validate environments mimicking massive IoT deployments.
7. **Time-sensitive Task Offloading:** In industries like Industry 4.0 and autonomous vehicles, where edge tasks must be processed instantly for timely decision-making, the ability to guarantee ultra-low latency is invaluable. RAP twin’s ability to perform dry-run evaluations ensures that organizations can confidently deploy algorithms capable of meeting strict, time-sensitive deadline constraints when they need it most.

The motivation behind RAP twin is not just about simulation or basic task scheduling—it’s about empowering network engineers and researchers to transform their approach to distributed edge orchestration. By removing the bottlenecks and risks created by static scheduling and live-infrastructure testing, RAP twin helps organizations become more resilient, efficient, and capable of guaranteeing strict Service Level Agreements (SLAs) under dynamic and unpredictable network conditions.

Ultimately, the goal is to help organizations in key sectors like Industry 4.0 smart manufacturing, Internet of Vehicles (IoV), and next-generation 5G/6G telecommunications unlock the full potential of advanced AI, federated, and Markov-based scheduling policies. This ensures heterogeneous workloads are optimally placed while significantly reducing the

operational costs, deployment risks, and hardware inefficiencies that have historically plagued complex edge computing environments

1.3 Literature Survey

Over the past decade, significant progress has been made in the development of Digital Twins (DTs) and the management of distributed computing systems. However, most existing systems are either domain-specific or lack the intelligent, integrated mechanisms necessary for handling the complex scheduling, unpredictable failures, and performance variability inherent in modern distributed computing fabrics. This literature review explores several notable contributions in the field and highlights the research gaps that RAP twin aims to address.

High-fidelity bidirectional data synchronization, resilient workload scheduling, and systematic fault-injection are fundamental challenges in managing distributed systems and microservice architectures. While significant research has been conducted in these domains—such as independent applications of chaos experiments to build confidence against turbulent conditions and the use of data-driven models for real-time monitoring and predictive maintenance current solutions often remain fragmented. They generally lack an integrated, end-to-end sandbox framework capable of safely simulating, injecting deterministic failures, and evaluating multiple placement policies simultaneously. The following review outlines key contributions from the academic literature and how RAP twin builds upon them to offer a comprehensive solution.

To develop a robust, high-fidelity digital-twin and planning sandbox for heterogeneous distributed infrastructures, an in-depth review of key research in the areas of Digital Twin reference architectures, Chaos Engineering in microservices, and artificial intelligence-driven system optimization was conducted. The reviewed literature highlights both existing solutions and the operational research gaps that directly inform the design, telemetry ingestion, and multi-strategy planning engine of RAP twin.

Mihai et al. [?] surveyed Digital Twin (DT) reference architectures, emphasizing the fundamental need for continuous, bidirectional data synchronization between physical entities and their virtual counterparts to enable real-time monitoring and predictive analytics. While theoretical frameworks, such as the 5-dimensional DT architecture, comprehensively map these interactions, they often lack practical, self-contained implementations specifically tailored for the complexities of heterogeneous distributed computing fabrics. *RAP twin* operationalizes these academic concepts through its core **dt.state.DTState module, which acts as a unified state machine**. By continuously merging static topology descriptors with live telemetry via the `/observe` endpoint, it ensures the planning engine always reflects real-time constraints, addressing the performance variability inherent in distributed systems.

Esen et al. [?] provided a systematic review of Chaos Engineering in microservice

architectures, highlighting tools like Chaos Monkey and Gremlin that inject failures into environments to build confidence against turbulent conditions. Although these industry-standard tools are highly effective, executing them directly in production environments carries inherent risks and limits the ability to safely pre-evaluate how new scheduling algorithms react to cascading outages. *RAP twin* bridges this gap by introducing an **integrated chaos engine that injects deterministic failures**—such as link outages, node failures, and thermal derates—into a high-fidelity simulated sandbox. This allows engineers to systematically validate system resilience and scheduling adaptations without jeopardizing live production workloads.

Ding et al. [8] explored the integration of advanced artificial intelligence, such as deep reinforcement learning, with DTs to enable dynamic and adaptive scheduling in smart industries. While models like RL-Markov offer sophisticated autonomous decision-making, their high computational costs and low interpretability make them exceedingly difficult to evaluate against traditional heuristics across varied infrastructure scenarios. *RAP twin* solves this evaluation difficulty with its **Multi-Strategy Planning Engine, which supports plug-gable scheduling policies** ranging from baseline greedy and cheapest-energy strategies to advanced federated and RL-Markov models. Coupled with a **Monte-Carlo simulation runner**, it orchestrates thousands of trials to produce statistically significant benchmarks, allowing researchers to quantify policy performance under uncertainty.

Iranshahi et al. [?] emphasized that modeling and simulation form the core of a DT, identifying the fusion of physics-based and data-driven models as critical for predicting real-time system behavior. However, building unified predictive views from siloed telemetry sources remains a significant operational challenge in complex distributed systems. *RAP twin* overcomes this by employing a **PredictiveAnalyzer that processes diverse telemetry streams**—such as CPU, RAM, VRAM availability, and network loss—to compute accurate, real-time performance forecasts. This unified predictive view exposes per-node utilization forecasts and failure windows, empowering the scheduler to make proactive, informed job placement decisions.

Although significant progress has been made in the integration of Digital Twins (DT) and edge computing, the surveyed literature collectively reveals critical gaps that *RAP twin* aims to address. Prior research has successfully demonstrated the theoretical potential of Artificial Intelligence, including Multi-Agent Deep Reinforcement Learning (MADRL) and federated learning, for optimizing heterogeneous task offloading and resource allocation. However, the safe evaluation of these complex scheduling policies remains a major hurdle; testing advanced, AI-driven algorithms directly on live production infrastructure is highly risky, expensive, and prone to catastrophic failures caused by environmental unpredictability. Furthermore, while Chaos Engineering is now widely utilized in microservice architectures to test software resilience by deliberately terminating processes or simulating network disruptions, there is a distinct lack of tools designed to systematically inject deterministic, physical-layer failures—such as thermal derates, link outages, and network jitter—into a

high-fidelity edge emulation environment. Additionally, the literature highlights the difficulty of managing the dynamic nature of 5G/6G distributed networks, where performance telemetry from diverse hardware sources often remains siloed, preventing the creation of unified predictive views of how a job will perform on a specific node.

In conclusion, while prior research has significantly advanced DT reference architectures, Chaos Engineering tools, and AI-driven predictive maintenance, RAP twin uniquely integrates these isolated elements into a single, cohesive platform. RAP twin addresses the aforementioned gaps by combining a pluggable multi-strategy planning engine—capable of executing baseline greedy models alongside advanced RL-Markov and federated policies—with robust Monte-Carlo simulation tooling for large-scale statistical benchmarking under uncertainty. Moreover, it overcomes siloed telemetry by utilizing a unified state machine driven by a bidirectional REST API, which continuously merges static hardware descriptors with real-time streaming observations. By incorporating a dedicated Chaos Engine to simulate unpredictable edge conditions alongside predictive analytics for forecasting node utilization and failure windows, RAP twin creates a comprehensive, high-fidelity digital-twin and planning sandbox ecosystem. This ultimately bridges the gap between theoretical AI scheduling models and practical distributed systems orchestration, empowering researchers and engineers to statistically guarantee workload resilience prior to deploying on expensive physical infrastructure.

Chapter 2

Problem Definition and scope

2.1 Problem Statement

To develop a high-fidelity digital twin and planning sandbox that can intelligently schedule and orchestrate heterogeneous workloads across complex distributed computing fabrics—such as Edge, Multi-Cloud, and Hybrid environments (including those with siloed telemetry, dynamic thermal conditions, and unpredictable link outages) without risking costly disruptions to live production infrastructure . This will improve the statistical evaluation of advanced task offloading policies and enable safe, large-scale benchmarking and resilient architectural decision-making.

2.1.1 Goals and objectives

RAP twin aims at resilient and intelligent distributed task offloading, state synchronization, and performance benchmarking through AI-powered Deep Reinforcement Learning, predictive analytics, and Chaos Engineering-backed fault tolerance in the following ways:

- **Goals**

1. Establish RAP twin as a premier high-fidelity sandbox and emulation tool for distributed computing fabrics in critical sectors like Industry 4.0 manufacturing, telecommunications (5G/6G), and the Internet of Vehicles (IoV).
2. Achieve industry-leading optimization standards, demonstrating significant reductions in job completion time, system cost, and energy consumption compared to traditional static scheduling methods.
3. Contribute to the safe and risk-free transition of theoretical AI models (such as Multi-Agent and Federated Deep Reinforcement Learning) into practical engineering by eliminating the risk of catastrophic failures on expensive physical infrastructure.
4. Develop a highly scalable and robust closed-loop platform capable of ingesting live

streaming telemetry and handling extreme network volatility while maintaining perfect twin parity.

• Objectives

1. **Enable Intelligent Multi-Strategy Planning:** To dynamically schedule and optimize heterogeneous workloads using a pluggable planning engine that supports advanced algorithms (RL-Markov, federated, resilient, greedy) tailored to strict latency, cost, and energy targets.
2. **Achieve Real-Time State Synchronization:** To continuously merge static hardware descriptors (YAML) with live streaming telemetry via a bidirectional REST API (/observe endpoint), maintaining a highly accurate, unified state machine of the physical edge computing fabric.
3. **Automate Predictive Analytics Self-Healing:** To actively forecast thermal derating, node utilization, and network loss rates using background controllers (e.g., SelfHealingController and ResourceGuardian) that dynamically manage resources and clean up expired reservations.
4. **Facilitate Deterministic Chaos Engineering:** To proactively inject simulated network noise, link outages, and thermal throttling into the digital environment, allowing researchers to evaluate how scheduling policies adapt and reroute under real-world unpredictability.
5. **Execute Large-Scale Statistical Benchmarking:** To orchestrate thousands of scheduling trials via Monte-Carlo simulation tooling, generating comprehensive statistical proofs (CSV reports) of workload resilience, energy consumption, and Service Level Agreement (SLA) hit rates.
6. **Provide Interactive Visualizations Dry-Runs:** To allow engineers to evaluate plans and predict placement outcomes without committing actual physical resources, featuring a real-time web dashboard for tracking topology health, job history, and side-by-side strategy comparisons.

2.1.2 Assumptions and Scope

• Assumptions

- ◇ **Telemetry Quality:** It is assumed that the input data, such as static hardware descriptors and live telemetry updates, accurately reflect the real-world state of the edge nodes and network links without excessive estimation deviations.
- ◇ **Data Availability:** The project assumes that organizations can provide the necessary workload descriptors and streaming telemetry overrides via the designated

REST API endpoints.

- ◇ **Tech Infrastructure:** The tool assumes that the target user or organization has the necessary IT infrastructure, such as a local workstation or a Docker-enabled environment, to host the Flask API, the dashboard, and the simulation ecosystem.
- ◇ **Algorithm Suitability:** The project assumes that theoretical Artificial Intelligence algorithms, such as Multi-Agent Deep Reinforcement Learning and Federated models, can effectively optimize task offloading when provided with an accurate virtual representation of the physical network.

• Scope

- ◇ The project focuses on developing a high-fidelity digital-twin and planning sandbox for simulating, scheduling, and orchestrating heterogeneous tasks across distributed computing fabrics.
- ◇ It aims to convert static hardware constraints and live telemetry into optimized job placement plans and statistical performance metrics, making it easier for engineers to manage dynamic edge environments.
- ◇ The system is designed to support domains heavily reliant on edge computing and next-generation architectures, such as Industry 4.0 smart manufacturing, the Internet of Vehicles (IoV), and 5G/6G telecommunications.
- ◇ Deterministic fault-injection mechanisms (Chaos Engine) and Monte-Carlo simulation tooling will be incorporated to statistically validate scheduling resilience and Service Level Agreements (SLAs) under network uncertainty.
- ◇ The project emphasizes real-time state synchronization by continuously merging static node descriptors with live telemetry updates via a bidirectional REST API.
- ◇ The focus will remain on task offloading and simulated policy evaluation (e.g., greedy, RL-Markov, federated), rather than the physical execution and deployment of the actual containerized workloads.
- ◇ Both an administrative backend (Flask API) and a user-facing interactive web dashboard for tracking topology health, predictive analytics, and job history will be developed as part of this project.

By focusing on these assumptions and clearly defining the scope, RAP twin aims to offer a robust solution to the problems of distributed task scheduling and policy evaluation, while minimizing the challenges and risks associated with testing directly on live production infrastructure.

Chapter 3

Methodology

3.1 Methodology

Refer to Fig. 3.1 which shows the complete block diagram explaining the overall process flow of the RAPtwin system.

1. **Physical Fabric & Telemetry Ingestion:**

Edge Servers, IoT Devices, and Static Hardware Descriptors feed live telemetry streams and static configuration into the system via the REST API Gateway (/observe).

2. **State Merge & Sandbox Synchronization:**

Incoming state updates are merged and forwarded into the RAP twin Digital Sandbox, keeping the digital replica synchronized with the physical infrastructure.

3. **Chaos Engineering & Background Controllers:**

The `ResourceGuardian`, `SelfHealingController`, and `Chaos Engine` run continuously to handle reservation cleanup, node reliability, and fault injection respectively.

4. **Unified State Management & Predictive Analytics:**

A centralized state machine aggregates cluster states while the `PredictiveAnalyzer` generates real-time forecasts fed into the planning and simulation pipelines.

5. **Multi-Strategy Planning & Dry-Run:**

Job Descriptors with SLA constraints are evaluated across Greedy, RL-Markov, and Federated scheduling policies in Dry-Run mode for safe outcome prediction.

6. **Monte-Carlo Simulation & Result Analytics:**

Synthetic workload profiles are simulated across multiple trials, with results analyzed for JCT, energy consumption, and SLA adherence.

7. **Visualization, Export & Job Placement:**

Outputs are exported as Azure DTDL and Kubernetes CRDs, monitored via a Flask Dashboard, and dispatched as an optimized job placement directive to the physical fabric.

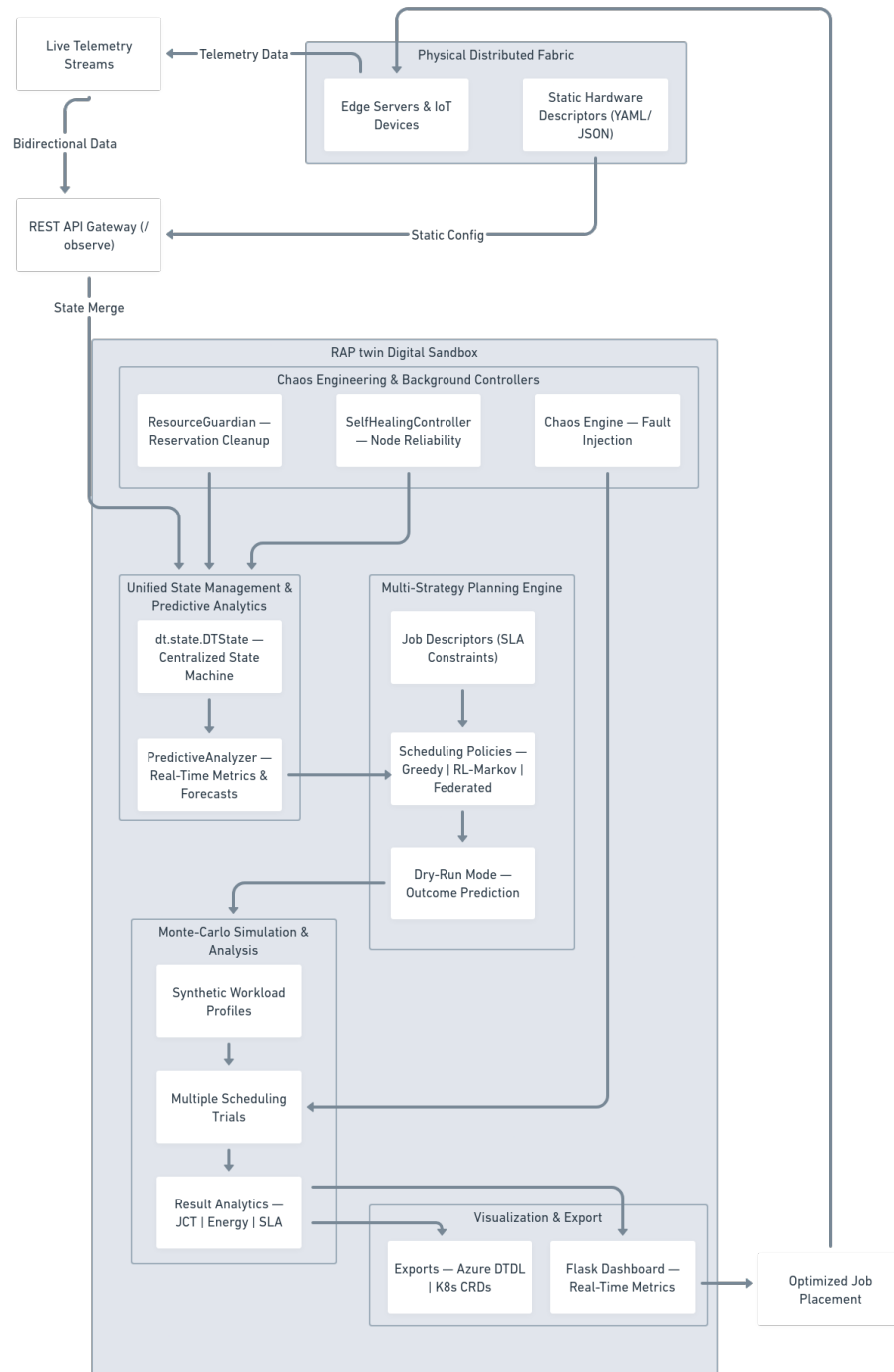


Figure 3.1: RAPtwin - Block Diagram

• Digital Twin Simulation & Planning Module

Refer to Fig. 3.2 which shows complete block diagram explaining the Digital Twin Simulation & Planning Module.

The RAP twin simulation and planning pipeline ensures high-fidelity, closed-loop emulation of distributed computing fabrics from raw telemetry ingestion to statistical performance evaluation using AI and Chaos Engineering technologies. The methodology includes state synchronization, workload planning, deterministic fault injection, predictive analytics, and final policy benchmarking. The entire methodology can be broken into the following sequential steps:

1. State Management & Telemetry Ingestion

- **Descriptor Parsing**

- Static hardware descriptors (YAML/JSON) containing node constraints and topology links are ingested and standardized.
- Job descriptors specifying SLAs, multi-stage workflows, and resource hints are parsed for the planner.

- **Live Telemetry Integration**

- The REST API (/observe endpoint) continuously merges streaming updates into a unified state machine to maintain perfect twin parity.
- Background self-healing controllers actively clean up expired reservations and monitor node reliability.

- **Predictive Analytics**

- AI-driven predictive modules forecast node utilization, thermal derating, and failure windows.
- Raw data is transformed into actionable metrics to evaluate risks before executing tasks on the physical fabric.

2. Multi-Strategy Workload Planning

- A pluggable planning engine assigns workloads using advanced algorithms (e.g., greedy, cheapest-energy, federated, RL-Markov).
- A dry-run evaluator predicts placement outcomes, allowing users to safely perform "what-if" analysis without committing actual computing resources.

3. Chaos Engineering & Fault Injection

- The Chaos Engine injects deterministic, physical-layer failures (e.g., link outages, network noise, thermal throttling) directly into the simulated environment.

-
- This ensures the scheduling policies can adapt and dynamically reroute workloads under real-world unpredictability.

4. Interactive Dashboard Preview

- Users are given a real-time web visualization of the topology maps, predictive health tracking, and job history.
- Based on the side-by-side strategy comparisons, users can adjust job constraints or manually apply observations (faults) to test system limits.

5. Statistical Evaluation & Exporting

- The Monte-Carlo orchestrator executes thousands of scheduling trials to statistically validate SLA hit rates, latency, and energy consumption.
- Output evaluation reports are generated in CSV format for bulk policy benchmarking.
- The final unified twin state can optionally be exported to industry-standard formats like Azure DTDL or Kubernetes CRDs for external control plane integration.

““



Figure 3.2: RAP twin - Simulation and Planning Pipeline

Chapter 4

Outcome

4.1 Type of Project

The RAP twin project is primarily application-oriented, with a focus on practical implementation and real-world problem-solving within the domain of distributed computing fabrics and edge orchestration.

- **Application-Oriented:**

The core goal of RAP twin is to create a functional, deployable sandbox that can be used by researchers and network engineers across sectors such as Industry 4.0 smart manufacturing, telecommunications (5G/6G), and the Internet of Vehicles (IoV) to automate and optimize job placement, scheduling, and performance analytics. The emphasis is on providing a solution that directly addresses the challenges faced by organizations dealing with highly dynamic, heterogeneous, and unpredictable edge computing environments. By leveraging a multi-strategy planning engine, Chaos Engineering, and predictive analytics, the project aims to enhance the resilience, efficiency, and scalability of computation offloading. It is designed for immediate practical use, offering a user-friendly interactive dashboard, CLI, and REST API that safely simulate closed-loop edge emulation without risking live production infrastructure.

- **Domain-Specific:**

- ◇ **Edge Computing & 5G/6G Networks:** RAP twin strengthens distributed infrastructure by safely managing and orchestrating heterogeneous workloads, dynamically addressing challenges like siloed telemetry, resource constraints, and time-sensitive task offloading.
- ◇ **Chaos Engineering & Simulation:** RAP twin leverages deterministic fault-injection and Monte-Carlo simulation tooling to create statistical benchmarks,

validating policy resilience and SLA hit rates under simulated real-world network uncertainty (e.g., link outages, thermal throttling).

- ◇ **Artificial Intelligence (AI):** RAP twin harnesses advanced AI—including Multi-Agent Deep Reinforcement Learning (MADRL), Federated Learning, and Markov Decision Processes (MDP)—to automate intelligent resource allocation and transform raw telemetry into actionable predictive insights.

- **Research-Oriented:**

While the primary focus of RAP twin is on application, the project also incorporates significant research elements in terms of translating complex, theoretical Artificial Intelligence models into a practical engineering environment. The tool is designed to push the boundaries of current computation offloading strategies, specifically focusing on handling the "estimation deviations" between a digital twin's virtual predictions and actual physical hardware states. Additionally, there are research aspects involved in developing sophisticated algorithms for real-time state synchronization, balancing exploration and exploitation in federated environments, and evaluating scheduling resilience through large-scale statistical validation. Therefore, the project also has an underlying research component aimed at bridging the gap between theoretical AI models and practical edge emulation for next-generation networks.

Chapter 5

Project Plan

5.1 Project Timeline

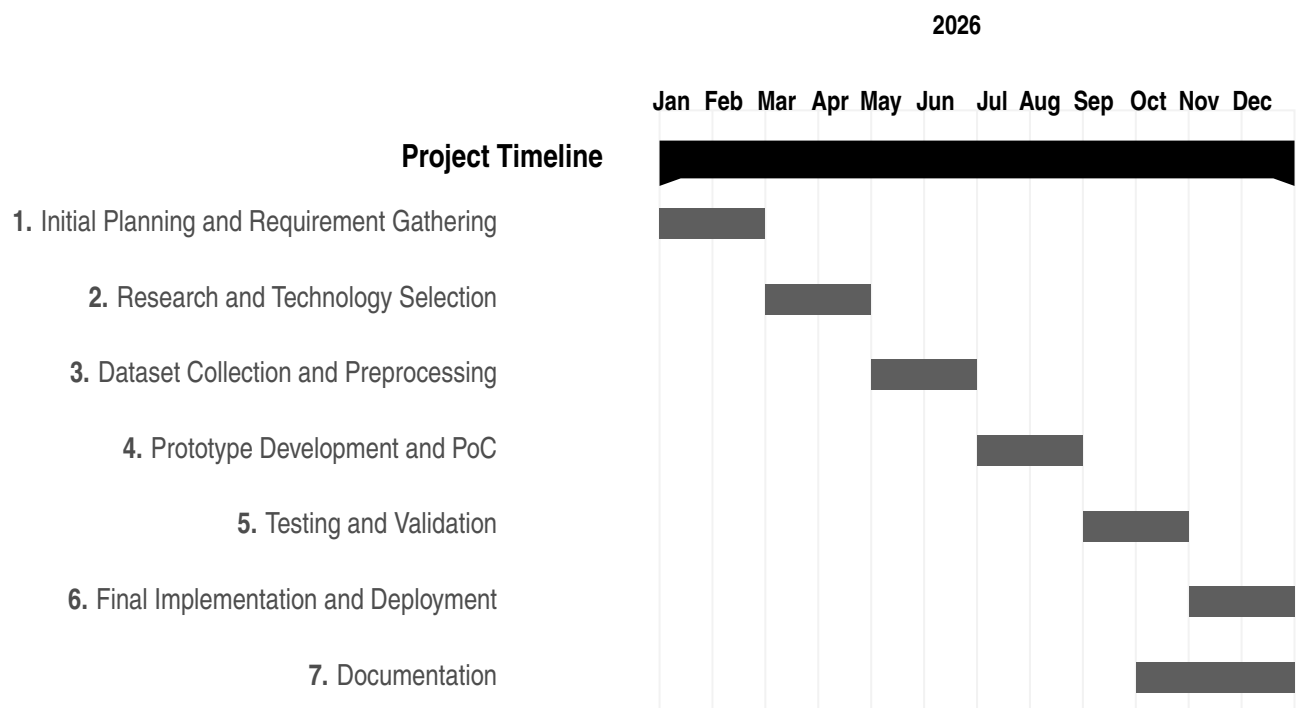


Table 5.1: Project Timeline

5.2 Team Organization

The project is carried out by a group of three team members, each assigned a specific role to ensure effective collaboration and clear reporting mechanisms.

5.2.1 Team Structure

1. Riaan – Digital Twin API & UI Dashboard

Riaan is responsible for the core Digital Twin infrastructure, state management, and user-facing visualizations. His tasks focus on ensuring that the virtual environment continuously mirrors the physical fabric and on building the interactive dashboard for researchers.

Key Responsibilities:

- ◇ Digital Twin API (Flask) backend development
- ◇ Unified State Machine (`dt.state.DTState`) implementation
- ◇ Live telemetry ingestion (`/observe` endpoint) and synchronization
- ◇ Background self-healing controllers (`SelfHealingController` & `ResourceGuardian`)
- ◇ Data encryption dashboard UI, including real-time topology maps and job history
- ◇ REST API security and data exporting (JSON, DTDL formats)

2. Avanti – Intelligent Planning & AI Scheduling

Avanti's responsibilities focus on the multi-strategy planning engine, workload constraint parsing, and implementation of advanced AI scheduling policies. She ensures that jobs are optimally placed across the fabric based on dynamically changing conditions.

Key Responsibilities:

- ◇ Multi-strategy planning engine development
- ◇ Federated Deep Reinforcement Learning (DRL) policy integration
- ◇ RL-Markov and MDP policy implementation
- ◇ Planner CLI development and dry-run evaluator
- ◇ Job descriptor parsing, SLA constraints, and validation (YAML/JSON)
- ◇ Baseline strategy implementation (Greedy, Cheapest-Energy, Bandit)

3. Preet – Simulation, Chaos Engineering & Analytics

Preet ensures that the system can be rigorously evaluated under real-world uncertainty. He is responsible for large-scale benchmarking, injecting network noise, and validating the resilience of the AI scheduling algorithms.

Key Responsibilities:

- ◇ Chaos engine development (deterministic fault and noise injection)
- ◇ Monte Carlo simulation tooling and orchestration
- ◇ Predictive analytics and failure forecasting (**PredictiveAnalyzer**)
- ◇ Synthetic workload and node generation
- ◇ Statistical policy evaluation and CSV reporting (metrics: latency, energy, risk)
- ◇ Docker-based fabric emulation and teardown scripts

Chapter 6

Experimental setup

In order to rigorously evaluate RAP twin, we established a high-fidelity controlled experimental environment that mirrors the dynamic, volatile conditions of a real-world distributed edge computing fabric. Below are the details of our setup, from the synthetic topology data we used to the performance metrics we tracked and the chaos engineering obstacles we simulated along the way.

6.1 Data Set

Synthetic Workloads and Edge Topologies

- **Source:** A collection of dynamically generated synthetic physical node descriptors and complex network topologies mimicking heterogeneous edge-end resources.
- **Formats:** Structured YAML and JSON files containing strict Service Level Agreement (SLA) deadlines, resource constraints, and predictive telemetry profiles.
- **Content Variety:**
 - ◊ Heterogeneous tasks including control, sensing (latency-sensitive, CPU-heavy), and multimedia operations (broadband, GPU-heavy).
 - ◊ Topology configurations spanning single-node execution, multi-edge collaboration, and cloud-edge hybrid networks.
 - ◊ Deterministic chaos profiles and live telemetry overrides (e.g., link jitter, thermal derating, and node failovers).

6.2 Technology Used

- **AI & Planning Engines:**
 - ◊ Pluggable multi-strategy planner utilizing Multi-Agent Deep Reinforcement Learning (MADRL), Federated Learning, and Markov Decision Processes (MDP).

-
- ◊ Actor-Critic neural network architecture with ϵ -greedy methods for dynamic policy optimization.
 - **Simulation & Chaos Tooling:**
 - ◊ Dedicated Chaos Engine (`sim/chaos.py`) for deterministic fault injection and network noise emulation.
 - ◊ Monte-Carlo orchestrator (`sim/montecarlo.py`) for thousands of large-scale statistical validation sweeps.
 - **Backend & Infrastructure:**
 - ◊ Python 3.7+ with a Flask-based REST API for handling live telemetry ingestion (`/observe`) and maintaining a unified state machine.
 - ◊ Web-based UI Dashboard rendering real-time topology maps and job histories.
 - **Development & DevOps:**
 - ◊ Git for version control and collaborative modeling.
 - ◊ Docker containers to ensure isolated, local fabric emulation and automated deployment tearing down scripts.

6.3 Performance Parameters

To measure success and uncover areas for improvement, we tracked:

- **Job Completion Time (JCT):**
 - Total latency to process heterogeneous tasks across edge-end computing resources.
 - Goal: Minimize JCT while adhering strictly to tasks' maximal deadline constraints.
- **Energy Consumption & System Cost:**
 - The weighted sum of communication/transmission energy and computing energy across all nodes.
 - Goal: Maximize system cost-efficiency and energy savings without sacrificing performance.
- **Scheduling Resilience & SLA Hit Rate:**
 - Percentage of workloads successfully placed and completed under unstable conditions.
 - Goal: High SLA adherence (>95%) despite simulated link outages, jitter, and thermal derating.
- **Policy Convergence & AI Rewards:**
 - Cumulative long-term reward tracking the MADRL agent's ability to balance exploration and exploitation.
 - Monitored to avoid local optimal traps and ensure algorithm stability during offline training.

- **Simulation Throughput (Monte-Carlo):**
 - Number of scheduling trials orchestrated per minute under uncertainty.
 - Target: Rapid evaluation generation yielding comprehensive statistical CSV reports to safely dry-run architectures.

Chapter 7

PO & PSO Mapping

7.1 Program Outcome (PO) Mapping

PO	Description	Level	Justification
PO1	Engineering knowledge: Apply the knowledge of mathematics, science, Engineering fundamentals, and an Engineering specialization to the solution of complex Engineering problems.	3	RAPtwin Applies mathematical modeling (MDP) and computer engineering principles to create a Digital Twin of heterogeneous clouds.
PO2	Problem analysis: Identify, formulate, review research literature and analyze complex Engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences and Engineering sciences.	3	Analyzed complex resource allocation bottlenecks and network variability to identify optimal scheduling points.

PO3	Design/development of solutions: Design solutions for complex Engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and Environmental considerations.	3	Designed a modular simulation platform including a thread safe state machine and a multi policy planning engine.
PO4	Conduct investigations of complex problems: Use research based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.	3	Extensive literature review and RL-model experiments were conducted
PO5	Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern Engineering and IT tools including prediction and modeling to complex Engineering activities with an understanding of the limitations.	3	RAPtwin utilizes modern development tools such as Python, Flask, YAML descriptors, and various AI/ML libraries.
PO6	The Engineer and The World: Analyze societal, environmental, sustainability impacts.	1	Addresses the need for energy efficient computing, which reduces the carbon footprint of large scale data centers.
PO7	Ethics: Apply ethical principles and commit to ethics and professional responsibilities.	3	Ensures data integrity in telemetry ingestion and follows ethical coding practices in software development.

PO8	Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.	3	Developed under mentorship and collaboration demonstrating both individual and team efforts.
PO9	Communication Skills: Communicate effectively on complex Engineering activities with the Engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.	3	Maintained clear project documentation, presentation (PPT), and technical reporting in the Achieve-Mind project.
PO10	Project management and finance: Apply project management and financial principles in engineering work.	2	The project involved planning for aspects such as cloud resource optimization, compute energy costs, and modular system integration; however, it did not encompass comprehensive financial management activities typical of a full scale engineering project.
PO11	Life long learning: Recognize the need for life long learning amidst technological change.	3	Built a scalable architecture that allows for continuous learning and addition of new AI models as technology evolves.

Table 7.1: Program Outcomes (PO) Mapping – RAPtwin

7.2 Program Specific Outcome (PSO) Mapping

PSO Number	Description	Mapping Level	Justification
PSO1	Interpret data, use software tools to conduct experiments, and apply AI and machine learning techniques to solve multi disciplinary problems.	3	Interprets real time telemetry data, utilizes software tools for chaos experiments, and applies RL-Markov techniques to solve infrastructure optimization problems.
PSO2	Apply standard practices, strategies, and models of Data Science to discover knowledge.	3	Applies standard practices in predictive analytics and uses MDP models to discover optimal placement patterns from historical resource data.

Table 7.2: Program Specific Outcomes (PSO) Mapping – RAPtwin

Chapter 8

Summary

RAP twin offers a robust, high-fidelity digital twin and planning sandbox for automating and optimizing job placement, scheduling, and performance analytics across distributed computing fabrics. By combining Multi-Agent Deep Reinforcement Learning (MADRL), Federated Learning, Chaos Engineering, and Monte-Carlo simulation tooling, it ensures both intelligent task offloading and system resilience. Through real-time state synchronization via a bidirectional REST API, predictive analytics, and dynamic self-healing controllers, the system not only handles complex, heterogeneous edge environments but also guarantees strict Service Level Agreements (SLAs), minimized latency, and fault tolerance. With my contributions in developing the multi-strategy planning engine, implementing the unified state machine, and designing the deterministic fault-injection environment, RAP twin ensures a highly accurate, risk-free emulation of physical network conditions. Additionally, the dry-run evaluation loop, interactive web dashboard, and large-scale statistical benchmarking workflow enhance user control and confidence, making RAP twin a complete and reliable platform for end-to-end distributed systems orchestration and policy validation.

References

- [1] Q. Qi, F. Tao, T. Zuo and A. Y. Nee, “A review of digital twins in smart industries,” in *IEEE Access*, vol. 7, pp. 14232-14252, Jan. 2019, doi: 10.1109/ACCESS.2019.2895206.
- [2] A. Fuller, Z. Fan, C. Day and C. Barlow, “Digital Twin: Enabling Technologies, Challenges and Open Research,” in *IEEE Access*, vol. 8, pp. 108952-108971, June 2020, doi: 10.1109/ACCESS.2020.2998358.
- [3] F. Geyer and S. Bondorf, “Proximity-based service discovery for distributed digital twin systems,” *IEEE Access*, vol. 9, pp. 12456-12470, 2021.
- [4] Z. Lv, D. Chen, R. Lou, and H. Song, “Digital Twins: A Survey on Enabling Technologies, Challenges, Trends, and Future Prospects,” *IEEE Communications Surveys & Tutorials*, vol. 24, no. 1, pp. 54-87, 2022.
- [5] M. Grieves and J. Vickers, “Digital twins: Recent advances and future directions in engineering fields,” in *Nature Reviews Methods Primers*, vol. 2, no. 1, pp. 1-18, Dec. 2022, doi: 10.1038/s43586-022-00103-y.
- [6] J. Wang, L. Ye and R. X. Gao, “A simulation-based digital twin model for data-driven decision optimization,” in *Journal of Manufacturing Systems*, vol. 62, pp. 245-258, Jan. 2022, doi: 10.1016/j.jmsy.2021.11.011.
- [7] S. S. Kim, Y. J. Lee and M. G. Lee, “Digital twin model with machine learning for real-time monitoring and predictive maintenance,” in *International Journal of Precision Engineering and Manufacturing*, vol. 23, no. 5, pp. 565-578, May 2022, doi: 10.1007/s40684-022-00431-w.
- [8] C. Xu, Z. Tang, H. Yu, P. Zeng and L. Kong, “Digital Twin-Driven Collaborative Scheduling for Heterogeneous Task and Edge-End Resource via Multi-Agent Deep Reinforcement Learning,” in *IEEE Journal on Selected Areas in Communications*, vol. 41, no. 10, pp. 3056-3069, Oct. 2023, doi: 10.1109/JSAC.2023.3310066.
- [9] Y. Dai, J. Zhao, J. Zhang, Y. Zhang and T. Jiang, “Federated Deep Reinforcement Learning for Task Offloading in Digital Twin Edge Networks,” in *IEEE Transactions on Network Science and Engineering*, vol. 11, no. 3, pp. 2849-2861, May-June 2024, doi: 10.1109/TNSE.2023.3338363.

-
- [10] A. M. V. V. Sai, C. Wang, Z. Cai and Y. Li, “Navigating the Digital Twin Network landscape: A survey on architecture, applications, privacy and security,” in *High-Confidence Computing*, vol. 4, no. 4, p. 100269, Dec. 2024, doi: 10.1016/j.hcc.2024.100269.
 - [11] S. M. Raza, R. Minerva, N. Crespi, M. Alvi, M. Herath and H. Dutta, “A comprehensive survey of Network Digital Twin architecture, capabilities, challenges, and requirements for Edge-Cloud Continuum,” in *Computer Communications*, vol. 236, pp. 165-192, Jan. 2025, doi: 10.1016/j.comcom.2024.11.018.
 - [12] H. Tran-Dang and D.-S. Kim, “Digital Twin-empowered intelligent computation offloading for edge computing in the era of 5G and beyond: A state-of-the-art survey,” in *ICT Express*, vol. 11, no. 2, pp. 167-180, Apr. 2025, doi: 10.1016/j.ict.2024.08.006.